



Security Assessment Final Report



Safe Policy Engine

August 2026

Prepared for Safe

Table of Contents

Project Summary.....	3
Project Scope.....	3
Project Overview.....	3
Protocol Overview.....	3
Assessment Methodology.....	4
Threat and Security Overview.....	5
Findings Summary.....	10
Severity Matrix.....	10
Detailed Findings.....	11
Low Severity Issues.....	12
L-01: DELEGATECALL can execute with an unchecked msg.value.....	12
L-02: Pending configuration roots never expire.....	14
L-03: Failed token calls can consume policy state.....	16
L-04: Guard callback entrypoints can commit policy state for unexecuted actions.....	18
L-05: Design limitation makes enforcing a policy impossible in certain cases.....	20
Disclaimer.....	21
About Certora.....	21

Project Summary

Project Scope

Project Name	Repository Link	Commit Hash	Platform
Safe Policy Engine	https://github.com/safe-research/policy-engine	bcb5032 (initial) 405ba1d (final)	EVM

Project Overview

This document describes the security review of **Safe Policy Engine**. The work was undertaken from **August 17th, 2026** to **August 25th, 2026**.

The following contract list is included in our scope:

- contracts/core/PolicyEngine.sol
- contracts/SafePolicyGuard.sol
- contracts/libraries/AccessSelector.sol
- contracts/libraries/SignatureExtension.sol

The team performed a manual audit of all the files in scope. During the manual audit, the Certora team discovered bugs in the code, as listed on the following page.

Protocol Overview

Safe Policies is an on-chain policy engine for Safe smart accounts. It provides fine-grained, deny-by-default access control across both owner- and module-authorized transactions by mapping each target, function selector, and operation to an extensible policy contract. The system supports stateful checks, delayed configuration changes, fallback policies, token and native-asset restrictions, module controls, co-signing, and MultiSend validation. Its core **SafePolicyGuard** must be installed as both the transaction guard and module guard to ensure consistent enforcement across every Safe execution path.

Assessment Methodology

Our assessment approach combines design level analysis with a deep review of the implementation to ensure that a protocol is secure, economically sound, and behaves as intended under realistic conditions.

At the design level, we evaluate the architecture, the economic assumptions behind the protocol, and the safety properties that should hold independently of a specific chain or environment. This process includes reviewing internal and cross protocol interactions, state transition flows, trust boundaries, and any mechanism that could be exploited to extract value, deny service, or alter core system behavior. At this stage, a focused threat modelling exercise helps identify key attack surfaces and adversarial capabilities relevant to the system. Design level issues often relate to incentive structures, governance implications, or systemic behavior that emerges under adversarial conditions.

Implementation analysis focuses on the concrete behavior of the code within the execution model of the target chain. This involves reviewing the correctness of logic, access control, state handling, arithmetic behavior, and the nuanced behaviors of the chain environment. Familiar classes of vulnerabilities such as reentrancy conditions, faulty permission checks, precision issues, or unsafe assumptions often surface at this layer. These findings require context aware reasoning that takes into account both the code and the architectural intent.

To support this analysis, the codebase is examined through repeated manual passes and supplemented by automated tools when appropriate. High-risk logic areas receive deeper scrutiny, invariants are validated against both design intent and actual implementation, and potential vulnerability leads are thoroughly investigated. Automated techniques such as static analysis, fuzzing, or symbolic execution may be used to complement manual review and provide additional insight.

Collaboration with the development team plays an important role throughout the audit. This helps confirm expected behaviors, clarify design assumptions, and ensure an accurate understanding of the protocol's intended operation. All findings are documented with clear reasoning, reproducible examples, and actionable recommendations. A follow up review is conducted to validate the applied fixes and verify that no regressions or secondary issues have been introduced.

Threat and Security Overview

The audit scope covered the following components:

- `contracts/SafePolicyGuard.sol`
- `contracts/core/PolicyEngine.sol`
- `contracts/libraries/AccessSelector.sol`
- `contracts/libraries/SignatureExtension.sol`

SafePolicyGuard integrates the policy engine with Safe's transaction and module guard callbacks. It checks owner-authorized and module-authorized executions, extracts optional policy context, restricts gas-related transaction fields, and manages immediate and delayed policy configuration.

PolicyEngine converts each transaction into an access selector derived from its target, function selector, and operation. It resolves an exact policy first and then an operation-specific fallback. Missing policies deny execution. During a check, it records the Safe and authenticated module identity so recursive checks remain restricted to the same Safe and authorization path.

AccessSelector packs and extracts the target address, function selector, and operation from a single word. Its representation is security-sensitive because policy selection depends entirely on the uniqueness and correctness of this encoding.

SignatureExtension extracts optional context appended to the Safe signature bytes. The extension provides framing and type identification but does not authenticate its payload. An executor can add, replace, remove, or corrupt the context without changing the owner-signed Safe transaction.

Roles and trust boundaries

Safe is trusted to authenticate owners and enabled modules, increment its nonce in the expected order, call the correct guard hooks, and report low-level execution results according to the reviewed Safe version.

Safe owners remain constrained by active policies. A compromised owner threshold can request new configurations and apply them after the configured delay. The delay provides a monitoring and response window rather than permanent protection against compromised owners.

Enabled modules are not globally trusted. Their transactions must pass through the module guard. The module identity supplied by Safe is treated as authenticated, while policy context is not.

Configured policy contracts are outside the code scope and are trusted to implement their authorization and accounting logic correctly. The scoped contracts deliberately permit policies to update state during pre-execution checks.

Executors and relayers are untrusted. They cannot modify the owner-signed Safe transaction fields, but they can choose the transaction submission time, the outer call value, and the trailing policy context.

Target contracts and external protocols are also outside the code scope. Their behavior may differ from the semantic action inferred from a selector. Examples include fallback functions, false-returning tokens, payable entrypoints, and delegated implementations that inspect the outer execution environment.

Expected invariants

- The guard is installed on every Safe execution path that is expected to be protected.
- A transaction without an exact or fallback policy is denied.
- Exact policies take precedence over fallback policies.
- **CALL** and **DELEGATECALL** policy namespaces remain separate.
- The target, selector, and operation encoded by **AccessSelector** do not overlap or corrupt one another.
- Empty calldata maps to the zero selector, while calldata between one and three bytes fails closed.
- A policy must return the expected magic value for execution to continue.
- The module identity received by a policy comes from the Safe callback and cannot be forged through context.

- A direct engine check outside an active guard callback is rejected.
- Recursive checks cannot switch from the currently checked Safe to another Safe.
- A policy cannot begin another top-level guard check while its own top-level check is active.
- State written during a policy check is rolled back when the corresponding low-level Safe execution reverts.
- Immediate configuration is unavailable once the guard is installed in either Safe guard slot.
- Delayed configurations cannot be applied before maturity or replayed after successful application.
- Signature context is treated as untrusted unless a policy independently authenticates it.
- An absent signature extension produces empty context.
- A malformed extension that claims the recognized type fails closed.
- Signature extension decoding cannot read outside the supplied signature bytes.
- The transaction tuple evaluated by the policy represents the effective behavior observed during execution.

Attack vectors considered

The review considered ordinary Safe transactions that target the guard's callback entrypoints. Such calls can be indistinguishable from authentic callbacks because both originate from the Safe. If authorized by a policy, an ordinary callback call can initiate a second policy check over attacker-selected parameters without executing the action described by those parameters. This is particularly relevant to stateful policies and to the module callback, where the ordinary call can supply an arbitrary module address.

Stateful policy accounting was reviewed against Safe's low-level execution result. Safe reports whether a call reverted, but it does not interpret application return data. A token that returns false without reverting can therefore cause policy state to commit even though no transfer or approval occurred. The resulting economic risk is exhaustion of a spending limit, counter, or one-time authorization without a corresponding asset movement.

The review also considered the difference between the value declared in a signed Safe transaction and the value attached to the outer `execTransaction` call. During `DELEGATECALL`,

delegated code observes the outer `msg.value`. An untrusted executor can choose this value even when the policy checks a signed declared value of zero. Approved delegated code can consequently follow a value-dependent execution path that was not represented by the checked tuple.

The delayed configuration lifecycle was examined for stale authorization. Configuration roots record the earliest application time but do not expire. A root can remain applicable across recovery, guard removal, and later reinstallation. Applying it still requires Safe authorization, but an undiscovered matured root removes the response window that would otherwise accompany a new request.

Policy selection was reviewed against target fallback behavior. Different calldata selectors can reach the same fallback function while resolving to different policy entries. A strict policy configured for one selector does not necessarily constrain an alternative selector that reaches equivalent behavior through the target's fallback function. This is a model limitation of selector-based enforcement and is relevant to contracts such as wrapped native-token implementations.

The main dependency surface was Safe's callback, signature, value-forwarding, and execution behavior. Cross-protocol risks arise where external tokens or applications define success differently from low-level EVM success, or where target fallback behavior does not match the function semantics assumed by the policy configuration. The scoped system does not contain pricing, oracle, lending, or liquidation logic. Its economic risks arise from unauthorized Safe effects, consumed policy capacity, stale configuration authority, and differences between checked inputs and effective execution inputs.

Review and test coverage

Manual review covered the owner and module guard flows, policy selection, fallback resolution, selector encoding, signature extension parsing, reentrancy confinement, stateful check rollback, immediate configuration, delayed configuration, root invalidation, and execution outcome hooks. Safe integration code was inspected only where necessary to validate the assumptions used by the four scoped components.

The current repository test suite completes with 121 passing tests. Focused reproductions were used to validate callback-driven phantom checks, state persistence after false-returning token calls, and unchecked outer value during `DELEGATECALL`.

No checked-in fuzzing harness, invariant runner, Certora specification, or other formal-verification artifact was identified. No quantitative line or branch coverage result was available. Notable gaps include permanent regression tests for every guard callback selector, property-based testing of signature extension bounds, invariant testing of configuration roots, broader false-returning target behavior, and compatibility testing across multiple Safe versions.

Issue summary and security takeaways

The identified issues are the following:

- Low: Design limitation makes enforcing a policy impossible for some fallback-based target behaviors.
- Low: Guard callback entrypoints can commit policy state for actions that are never executed.
- Low: Pending configuration roots never expire.
- Low: Failed token calls can consume stateful policy capacity when failure is reported by returning false.
- Low: DELEGATECALL can execute with an unchecked outer msg.value.

No Critical, High, or Medium-severity issues have been identified.

The main security takeaways are to restrict ordinary transactions targeting guard callbacks, distinguish attempted actions from application-level success when designing stateful policies, limit delegated execution to implementations whose behavior is independent of unchecked environmental inputs, add expiry or bulk cancellation for pending configuration roots, and account for target fallback behavior when defining selector-based policies.

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	–	–	–
High	–	–	–
Medium	–	–	–
Low	5	5	2
Informational	–	–	–
Total	5	5	2

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
Likelihood				

Detailed Findings

ID	Title	Severity	Status
L-01	DELEGATECALL can execute with an unchecked msg.value	Low	Acknowledged
L-02	Pending configuration roots never expire	Low	Fixed
L-03	Failed token calls can consume policy state	Low	Acknowledged
L-04	Guard callback entrypoints can commit policy state for unexecuted actions	Low	Fixed
L-05	Design limitation makes enforcing a policy impossible in certain cases	Low	Acknowledged

Low Severity Issues

L-01: DELEGATECALL can execute with an unchecked msg.value

Severity: Low	Impact: Medium	Likelihood: Low
Files: contracts/SafePolicyGuard.sol contracts/core/PolicyEngine.sol	Status: Acknowledged	

Description:

`SafePolicyGuard` receives the value declared in an owner-authorized Safe transaction and forwards it through `PolicyEngine` to the selected policy. A policy therefore makes its decision using this declared value:

```
checkTransaction(msg.sender, to, value, data, operation,  
_decodeContext(signatures));  
  
IPolicy(policy).checkTransaction(safe, to, value, data, operation, ...);
```

For a `DELEGATECALL`, however, the declared value is not the value observed during execution. Safe executes the operation with the EVM `delegatecall` instruction, which has no value argument and preserves the `msg.value` of the current Safe execution frame. `Safe.execTransaction` is payable, while the ETH attached to that outer call is not included in the Safe transaction hash signed by the owners.

Consequently, an untrusted relayer can submit an unchanged, valid owner-signed transaction while choosing the value that delegated code observes:

1. The owners sign a transaction with operation set to `DELEGATECALL` and declared value equal to zero.
2. The relayer submits the valid transaction and attaches a nonzero amount of ETH to `Safe.execTransaction`.

3. `SafePolicyGuard` and the selected policy check the signed zero value and approve the transaction.
4. The delegated implementation executes in the Safe context with `msg.value` equal to the unsigned amount attached by the executor.

The attached ETH is credited to the Safe rather than the implementation address, but the delegated code can branch on the nonzero `msg.value`. If an approved implementation uses `msg.value` to choose a recipient, accounting path, asset transfer, or privileged action, the executor can trigger behavior that was not represented by the tuple checked by the policy. Since delegated code executes with the Safe's storage, balance, and authority, this can result in loss of Safe assets or unauthorized Safe configuration changes.

Recommendations:

Consider some of the following alternatives:

1. Modify the Safe integration or use a dedicated execution entry point that exposes the actual outer `msg.value` to `SafePolicyGuard`. The guard should pass this value to the selected policy so that the policy can compare it with the declared value or enforce that it is zero before approving a `DELEGATECALL`.
2. Use the `msgSender` parameter provided by Safe instead of discarding it. `SafePolicyGuard` can restrict owner-authorized transactions to an approved executor, or forward the executor identity to the selected policy. This alternative is effective only when the trusted executor guarantees that the outer `msg.value` matches the expected value.
3. If the limitation is accepted, document that the value supplied to policies is only the declared Safe transaction value and is not the effective `msg.value` observed during `DELEGATECALL` execution.

Customer Response:

Acknowledged and documented in [1ba723e](#).

L-02: Pending configuration roots never expire

Severity: Low	Impact: Medium	Likelihood: Low
Files: contracts/SafePolicyGuard.sol	Status: Fixed	

Description:

`SafePolicyGuard` implements delayed policy changes by storing the earliest timestamp at which each configuration root may be applied. However, the stored value has no expiration time:

```
rootConfigured[msg.sender][configureRoot] = block.timestamp + DELAY;

require(rootConfigured[msg.sender][configureRoot] != 0,
RootNotConfigured(configureRoot));

require(block.timestamp >= rootConfigured[msg.sender][configureRoot],
RootConfigurationPending());

delete rootConfigured[msg.sender][configureRoot];
```

Once the delay has elapsed, the root remains applicable indefinitely unless the Safe applies or individually invalidates it. Removing `SafePolicyGuard` from the Safe does not clear the mapping because it is stored in the guard contract, and reinstalling the same guard address does not establish a new configuration epoch. The contract also provides no bulk invalidation mechanism or on-chain enumeration of pending roots.

This creates the following recovery risk:

1. An attacker temporarily compromises the Safe's owner threshold and requests a malicious configuration root.
2. The legitimate owners recover control before the attacker applies the root, but fails to identify and invalidate it.

3. The root matures and remains stored, including across removal and later reinstallation of the guard.
4. If the attacker compromises the threshold again, it can apply the old root immediately instead of waiting through a new delay.

A stale root is therefore a latent policy-change capability that can bypass the monitoring and response window expected for a new configuration request. Applying it still requires authorization by the Safe, but incident recovery depends on discovering and individually invalidating every historical root. A missed root can later install permissive policies, authorize asset transfers, or enable changes to privileged Safe configuration.

Recommendations:

Give requested roots a bounded application window and reject them after it expires.

Customer Response:

Fixed in [dcf6841](#).

Fix Review:

Fixed.

L-03: Failed token calls can consume policy state

Severity: Low	Impact: Medium	Likelihood: Low
Files: contracts/SafePolicyGuard.s ol contracts/core/PolicyEngine. sol	Status: Acknowledged	

Description:

PolicyEngine invokes a policy before Safe executes the target transaction, and **IPolicy** explicitly permits **checkTransaction** to mutate state. This allows custom policies to implement spending limits, rate limits, counters, and one-time nonces. **SafePolicyGuard** attempts to make these updates atomic by rejecting nonzero **safeTxGas** and reverting in its after-execution hooks when Safe reports failure.

However, Safe's success flag only indicates whether the low-level **CALL** or **DELEGATECALL** reverted. It does not reflect the meaning of the target's return data. Some ERC-20 tokens report a failed transfer or approval by returning false without reverting. Safe treats such a call as successful and invokes the after-execution hook with success equal to true, so **SafePolicyGuard** does not revert the transaction.

This produces the following sequence:

1. A stateful policy updates a spending limit or consumes a policy nonce during its pre-execution check.
2. Safe calls the target token.
3. The token returns false without changing balances or allowances.
4. Safe classifies the non-reverting call as successful.
5. The transaction completes and the policy update remains committed even though the intended token action did not occur.

An attacker able to submit an otherwise authorized transaction can repeatedly consume policy capacity or one-time authorizations without performing the intended action, potentially

preventing later legitimate transactions. The same mismatch can also consume the Safe transaction nonce.

Recommendations:

Route calls to false-returning tokens through an adapter that reverts when the token returns false.

Also document that stateful policies account for attempted calls, not necessarily successfully completed actions.

Customer Response:

Acknowledged and documented in [b32b972](#).

L-04: Guard callback entrypoints can commit policy state for unexecuted actions

Severity: Low	Impact: Medium	Likelihood: Low
Files: contracts/SafePolicyGuard.s ol contracts/core/PolicyEngine. sol	Status: Fixed	

Description:

Safe invokes the transaction and module guard callbacks with the Safe as `msg.sender`. However, `msg.sender` has the same value when the Safe executes an ordinary CALL whose target is `SafePolicyGuard`. The callback entrypoints treat either call as an authentic pre-execution callback and start a new top-level policy check:

```
_enterCheck(msg.sender, address(0));  
checkTransaction(msg.sender, to, value, data, operation,  
_decodeContext(signatures));  
_exitCheck();
```

An ordinary transaction to one of these callbacks is first checked normally. If an exact policy or permissive fallback authorizes the outer call, the Safe then calls the callback as its transaction target. By this point the original guard check has exited, so the callback can enter a fresh check using attacker-chosen arguments. `PolicyEngine` resolves and invokes the policy using those inner transaction parameters, but no corresponding inner action is ever executed.

The module callback also treats its module argument as authenticated. During a genuine callback the Safe supplies this value, but during an ordinary call the transaction author chooses it. The second policy check can therefore pass a forged module identity to a policy.

For example, an authorized transaction can exploit the issue as follows:

1. A stateful policy is configured for a protected target, and a policy or fallback permits calls to a `SafePolicyGuard` callback selector.

2. The Safe executes a transaction whose actual target is `SafePolicyGuard.checkTransaction` or `SafePolicyGuard.checkModuleTransaction`.
3. The calldata describes a call to the protected target that will not be executed and, on the module path, an arbitrary module address.
4. The selected policy updates its state while checking the fake call.
5. The callback returns successfully, so the outer Safe transaction succeeds and the policy update is committed even though the described action never occurred.

Because `IPolicy.checkTransaction` explicitly supports state mutations, this can consume a spending limit, burn a nonce or one-time authorization, or trigger another persistent policy transition without performing the action that the policy state is intended to track. A forged module identity can also corrupt module-specific accounting or authorization state. Depending on the policy, the result can be a persistent denial of service or an application-specific security violation.

Recommendations:

Before resolving a policy, reject any `CALL` or `DELEGATECALL` to the guard's callback functions. Block both `checkTransaction` selectors, `checkModuleTransaction`, `checkAfterExecution`, and `checkAfterModuleExecution`. Direct callbacks from the Safe should remain unaffected.

Customer Response:

Fixed in [O22d96b](#).

Fix Review:

Fixed.

L-05: Design limitation makes enforcing a policy impossible in certain cases

Severity: Low	Impact: Medium	Likelihood: Low
Files: contracts/core/PolicyEngine. sol	Status: Acknowledged	

Description:

Policies are keyed by target, selector and operation and if one does not exist, the code routes to a per-safe fallback policy. The issue is for certain contracts which have a fallback function, i.e. WETH:

```
function() public payable {  
    deposit();  
}
```

Imagine there is a strict policy set based on **WETH**, **deposit()**, **CALL**. A caller can simply do a call to WETH with any random calldata/selector which will bypass the intended selector, but still run the deposit function via the fallback. In such case, the fallback policy will execute which can be more permissive.

Recommendations:

There are 2 main options:

1. Consider this as a design limitation and document it.
2. Add a second wildcard policy fallback case, running before the current fallback policy, where we only key based on target and operation. In this case, the configuration would be a strict policy on the **WETH**, **deposit()**, **CALL** key and then the wildcard policy on **WETH**, **CALL** would, for example, reject all calls routing via there.

Customer Response:

Acknowledged and documented in [df10c30](#).

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline and is widely used by developers and security researchers during audits and bug bounties.

Certora also provides architectural design reviews, where researchers analyze system design and trust boundaries early to identify structural risks and reduce complexity before implementation.

Certora conducts off-chain audits covering backend services, APIs, frontend, and mobile applications, focusing on vulnerabilities in authentication, data handling, key management, and wallet-related workflows.

Certora also offers Penetration Testing and Red Teaming to simulate real-world attacks and uncover exploitable weaknesses across infrastructure, applications, and business logic.

In addition, Certora provides operational security (OpSec) assessments, reviewing key management, access controls, and operational processes to strengthen overall security posture.

Finally, Certora delivers ongoing monitoring and incident response, enabling continuous threat detection and prevention, alert triage, and coordinated response to active incidents.